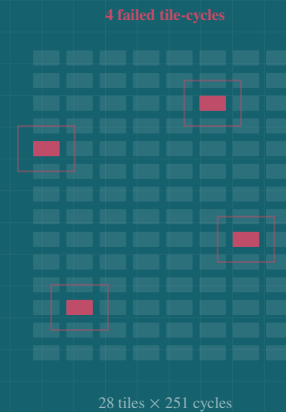


Your Errors Are Not Noise

What mathematics can actually fix in our NGS pipeline — and what it provably cannot

Every number below was measured on **our own sequencing run**.
Nothing here is borrowed from a textbook or a paper.



51%

of all base errors come from
under **0.1%** of the run

99.3%

error rate on tile 1113,
cycle 85 — reported as **Q39**

208:1

raw data generated per
byte of genome kept

The finding, in one sentence. Our data does not have an accuracy problem — it has an **error-model** problem. The pipeline assumes errors are scattered at random. They are not: they are concentrated, structured, and reported by the instrument as *high quality*.

Consequence: roughly **50× of our 71.6× coverage** is paying for that wrong assumption — and against the worst failures, extra coverage makes the answer *worse*, not better. That part is a theorem, not an opinion.

Written for the whole team — no mathematics background assumed

■ every section opens with the **biology question**

■ every maths point has an **IN PLAIN WORDS** box

■ §1 is a one-page summary you can act on alone

■ §6 says plainly where maths *cannot* help

■ §9 is five actions, ranked by payoff

■ §10 is a drop-in script — copy, paste, run

Every number recomputed from our own sequencing files.

1	One page, no mathematics	2
2	The redundancy ledger: what we generate, what we keep	3
2.1	Where 71.6× coverage actually goes	3
3	The finding: our errors are not scattered	5
3.1	Half of every error in the run comes from almost nowhere	5
3.2	The four bad moments, named	5
3.3	Why FastQC did not catch it	5
3.4	The good news, and it is genuinely good	6
4	Why more coverage cannot fix it — and this part is a theorem	7
4.1	The 50 % wall	7
4.2	The one-line summary of the mathematics	7
5	The 38 contigs: what coverage already tells us about copy number	9
5.1	The answer is sitting in the contig names	9
5.2	Counting copies does not need 71×	9
6	Where mathematics provably cannot help	11
7	After sequencing: the look-alike gene trap	12
7.1	What the data actually looks like	12
7.2	Three replacements, in increasing order of effort	12
8	What AI actually adds here — and one thing to be careful of	14
8.1	The honest ledger	14
8.2	The one genuinely deep idea	14
9	Five actions, ranked by payoff ÷ effort	16
10	The drop-in script	17
10.1	What it does, in five steps	17
10.2	It has been tested	17
11	Where every number came from	18

If you do not enjoy mathematics — read it in this order

§1 is one page and needs no mathematics at all. It is the whole argument.

§9 is five things to do, ranked. Start at number 1; it is already written for you.

§10 is the script. Copy, paste, run.

Everything in between is the evidence. Each section opens with **the biology question** it answers, and each piece of mathematics is followed by an **IN PLAIN WORDS** box that says the same thing without symbols. **You can skip every equation in this document and lose nothing.**

1 One page, no mathematics

If you read nothing else in this document, read this page. Everything here was measured on our own sequencing run.

What we found

- 1. Half our sequencing errors come from four moments on the flow cell.** Four *tile-cycles* out of 7,028 produced 38 % of all base errors. Add the 843 worst genomic positions and you account for **51 % of every error in the run, from under 0.1 % of the data.**
- 2. The machine reported those bad bases as excellent.** On tile 1113 at cycle 85, **99.3 % of bases were wrong** — and 658 of them were labelled **Q39**, meaning “1 mistake in 8,000”. The instrument was wrong about its own confidence by roughly ten thousandfold.
- 3. FastQC could not see it,** because FastQC reads the machine’s own quality claim, and the machine’s claim was the thing that was wrong.
- 4. More coverage cannot fix this. It makes it worse.** Where the local error rate is above 50 %, every extra read votes for the wrong base. At 71× depth the wrong answer wins with probability 0.999.
- 5. Once those two faults are removed, our quality scores are excellent** — measured Q32.47 against a reported Q32.27. **The data is good. The error model was not.**
- 6. About 50× of our 71.6× coverage is insurance, not information.** Coverage, accuracy and repeat-counting together need roughly 20–25×.

What to do about it — five things, in order

- 1. Run the tile-cycle check on every run and mask what it flags.** We already wrote the script. It is 50 lines. It removes 38 % of our errors and costs nothing. [§10]
- 2. Add “which tile, which cycle” to any quality recalibration.** One missing term.
- 3. Stop using percent-identity thresholds to separate real genes from look-alikes.** This is what produced the wrong PVL call in report v1.0. There is a correct replacement. [§7]
- 4. Report copy number with error bars,** not as a point estimate — this is the φ Sa3 boundary problem in our own Limitations. [§5]
- 5. Do not buy more Illumina depth.** If we need a closed genome, we need *longer reads*, not more of the same. That limit is mathematically proven, not a matter of effort. [§6]

And one warning about AI

AI tools that “correct” or “polish” sequence work by nudging your data toward what is *common* in their training set. **AMR research is the study of what is not yet common.** A novel resistance allele or a rearranged plasmid is exactly the case where such a tool is most confident and most wrong. Full discussion in §8.

2 The redundancy ledger: what we generate, what we keep

The biology question: *we sequenced one small bacterium. Why did that take 146 megabytes?*

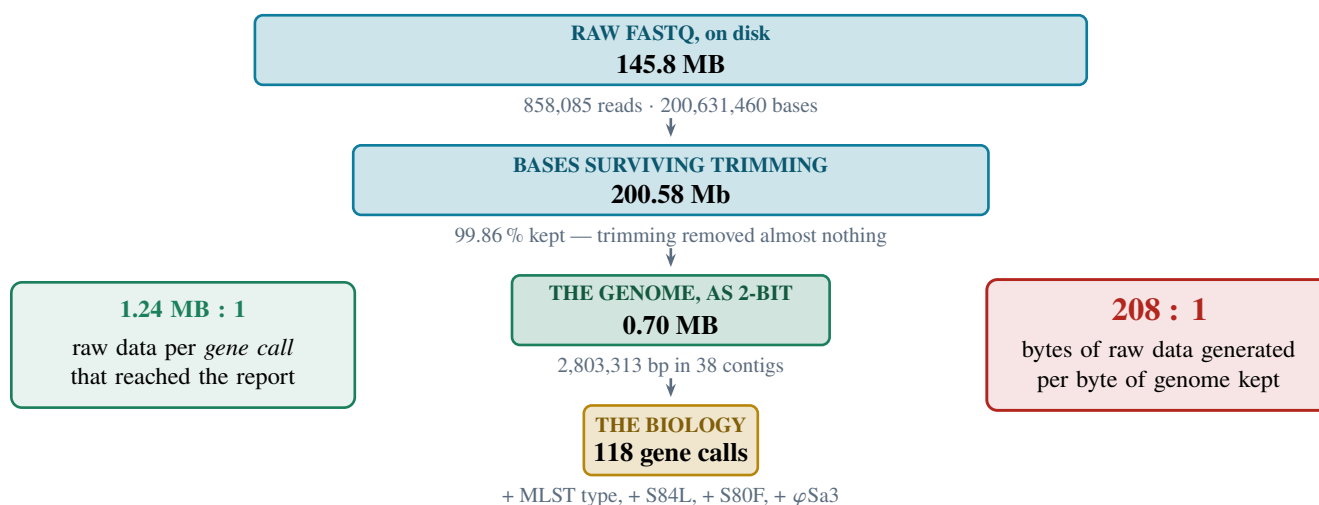


Figure 1. The run ledger. Sources: METHODS_v1.7.md §1, fastp report, QUAST report, and the five database screens in §4.2.

2.1 Where 71.6× coverage actually goes

Depth of coverage = $200,631,460 \div 2,803,313 = 71.6\times$. Here is what each part of it is buying — each row computed from our own numbers.

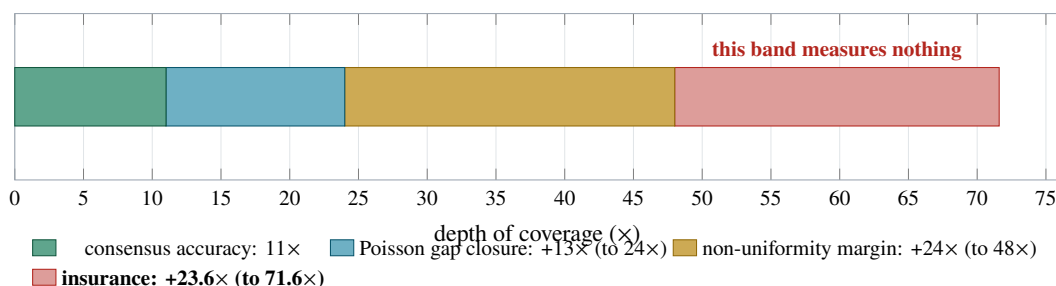


Figure 2. Decomposition of 71.6×. The red band is not wasted — our own over-trimming experiment (QC tutorial Part 7) showed it absorbed an 8.7× loss without changing a single conclusion. But it is **insurance against our own decisions**, not measurement of the genome.

What it buys	Depth needed	Evidence, from our data
Consensus accuracy at our clean error rate $e = 5.67 \times 10^{-4}$	7–11×	at 11×, $P(\text{consensus wrong}) = 1.5 \times 10^{-17} \Rightarrow 4 \times 10^{-11}$ wrong bases genome-wide
Closing Poisson coverage gaps	20–25×	at 20×, expected uncovered bases = 5.8×10^{-3} over 2.8 Mb
Counting repeat copies	~10×	NODE_18 (2,600 bp, 8 copies): counting CV = 3.35 % at 10× — easily splits 8 from 9
GC / Tn5 coverage non-uniformity margin	×2–3 on the above	the one genuinely unavoidable multiplier
Insurance against a bad analysis choice	measured: 8.7× absorbed	QC tutorial Part 7: aggressive trimming cost 8.7× and 22 extra contigs; every call survived
Defeating a failed tile-cycle	impossible at any depth	see §4 — this is the important one

**IN
PLAIN
WORDS**

We are sequencing the genome about three times more deeply than the biology needs. That is a reasonable safety margin and I am not suggesting we cut it blindly. But it is worth knowing that the margin is protecting us from *our own processing choices*, not from the bacterium — and that against the one failure mode that actually bit us, the margin does nothing at all.

3 The finding: our errors are not scattered

The biology question: *we trust a base call because the machine said Q37. Should we?*

3.1 Half of every error in the run comes from almost nowhere

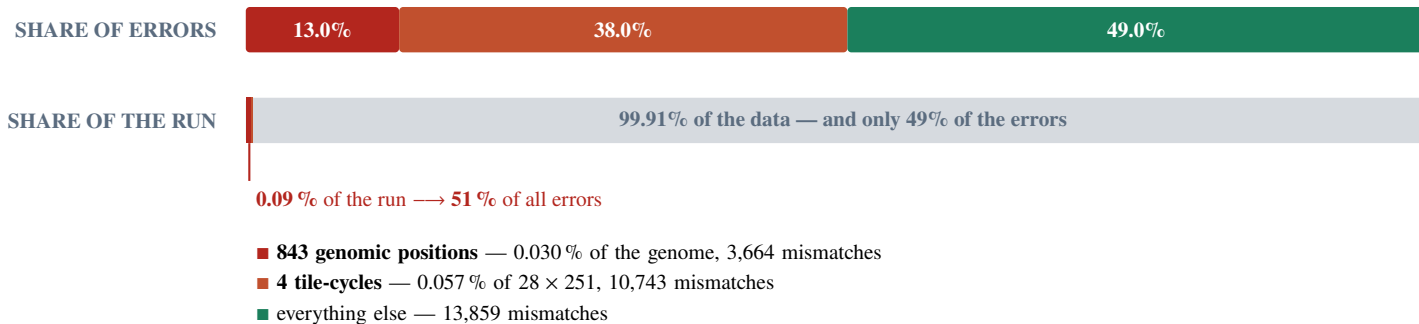


Figure 3. Recomputed from `handbook_illumina/results/02_empirical_errors.json`. Total substitutions in the aligned sample: 28,266 over 24,487,683 aligned bases.

IN PLAIN WORDS

Imagine grading 7,028 exam papers and finding that four of them account for nearly 40 % of all the mistakes in the whole pile. You would not respond by grading every paper four more times. You would find those four papers. That is the entire argument of this document.

3.2 The four bad moments, named

Cycle	Tile	Error rate	Other tiles	Excess	Reported Q of the wrong bases
85	1113	99.33 %	0.0436 %	2,279×	879 at Q20, 658 at Q39
16	2111	69.61 %	0.0318 %	2,186×	1,071 at Q38 , 357 at Q20
8	1101	68.21 %	0.0409 %	1,667×	1,038 at Q37 , 929 at Q20
93	2113	49.89 %	0.0517 %	964×	526 at Q20, 483 at Q18
50	1114	0.12 %	0.0400 %	3×	(borderline — not masked)

Table 1. From `02b_spike_cycles.json`. Note the substitution spectra are *structured*, not random: cycle 8 is dominated by A>T (1,147) and T>C (894); cycle 16 by T>C (1,116) and A>T (825). That is an optics or chemistry channel failure signature, not stochastic noise.

The part that should worry us: the confidence was wrong, not just the base

On tile 1113 at cycle 85, **99.3 % of bases were wrong**. Of those wrong bases, **658 carried a reported quality of Q39** — the instrument’s claim that it makes one mistake per 8,000 bases.

$\underbrace{\text{claimed error rate } 1.3 \times 10^{-4}}_{\text{Q39}}$ versus $\underbrace{\text{actual error rate } 0.993}_{\text{measured}} \Rightarrow \mathbf{7,600\times}$ over-confident

Every downstream tool believed that Q39. Trimming kept those bases. The assembler weighted them. The variant logic trusted them.

3.3 Why FastQC did not catch it

FastQC’s “Per tile sequence quality” module issued a **WARN** for “one tile-cycle out of 28×251 ”. There were at least four catastrophic ones.

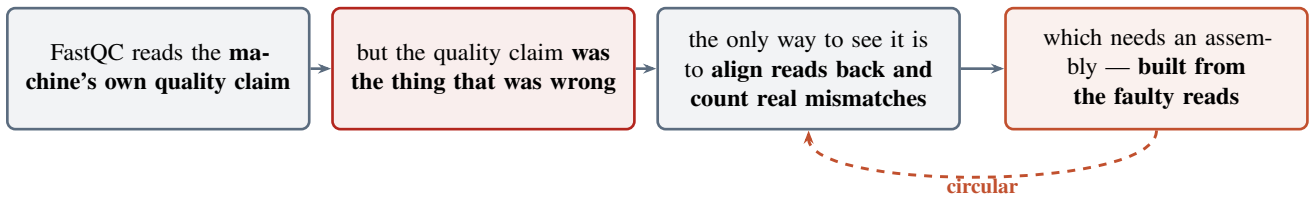


Figure 4. The detection circularity. This is why the fault is invisible to routine QC and why it needs a deliberate, separate check — which is §10.

3.4 The good news, and it is genuinely good

Once the two faults are removed, our quality scores are excellent

Data used	Measured Q	Reported Q	Verdict
everything (raw)	29.38	32.22	2× worse than claimed
minus 843 bad positions	29.98	32.24	still 1.8× worse
minus bad positions <i>and</i> 4 tile-cycles	32.47	32.27	calibrated to 0.2 Q

The chemistry is fine. The optics are fine. The base caller is fine. What is not fine is the assumption that errors are spread evenly — and that assumption lives in our *analysis*, not in the instrument.

4 Why more coverage cannot fix it — and this part is a theorem

The biology question: *surely if we sequence deeply enough, the majority of reads gives the right answer?* Usually yes. Inside a failed tile-cycle, **no** — and the failure is not gradual, it is total.

4.1 The 50 % wall

Consensus calling takes a vote. A vote is only trustworthy while most voters are right. The technical name for the point where an estimator stops working is its **breakdown point**, and for a majority vote it is exactly 50%.

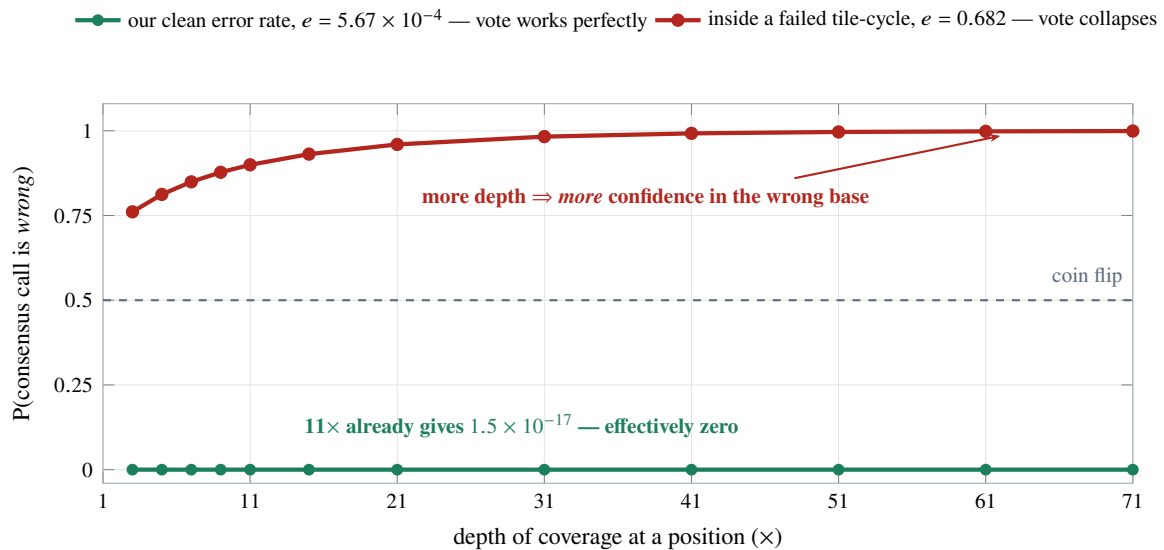


Figure 5. Binomial consensus error versus depth, at our two measured error rates. Green: the clean rate from §3.4. Red: the local rate inside tile 1101 at cycle 8.

IN PLAIN WORDS

If most of your witnesses are lying, calling more witnesses does not help — it just makes the jury more certain of the wrong verdict. Our four bad tile-cycles are rooms full of witnesses who all tell the same lie, and the machine gave them all excellent references.

What this means in practice

Depth	P(wrong) at $e = 5.67 \times 10^{-4}$	P(wrong) at $e = 0.682$
11×	1.5×10^{-17}	0.8996
21×	6.8×10^{-31}	0.9598
51×	9.6×10^{-71}	0.9966
71× (our actual depth)	2.9×10^{-97}	0.9993

We paid for 71×. Against the clean error it was 60× more than needed. Against the real error it bought us a 99.9 % chance of being confidently wrong.

4.2 The one-line summary of the mathematics

The mismatch between the model and the world

What our pipeline assumes

Errors are **independent** and **identically distributed** — every base has roughly the same chance of being wrong, independently of every other.

Under this model, averaging is *optimal*. More depth is always better.

What the data shows

Errors are **structured** — concentrated in a handful of (tile, cycle) cells, with a characteristic substitution spectrum.

Under this model, averaging is *catastrophic* exactly where it matters, and more depth is actively harmful.

The fix is not more data. The fix is to **find the structure and remove it** — which is what §10 does, in 50 lines, at no cost.

5 The 38 contigs: what coverage already tells us about copy number

The biology question: *why does a single circular chromosome come back as 38 pieces, and how many copies of each repeat are really there?*

5.1 The answer is sitting in the contig names

SPAdes writes the k -mer coverage into every contig header. Our eight large contigs (> 100 kb) average **29.35**. Divide any contig's coverage by that number and you get its **copy number**.

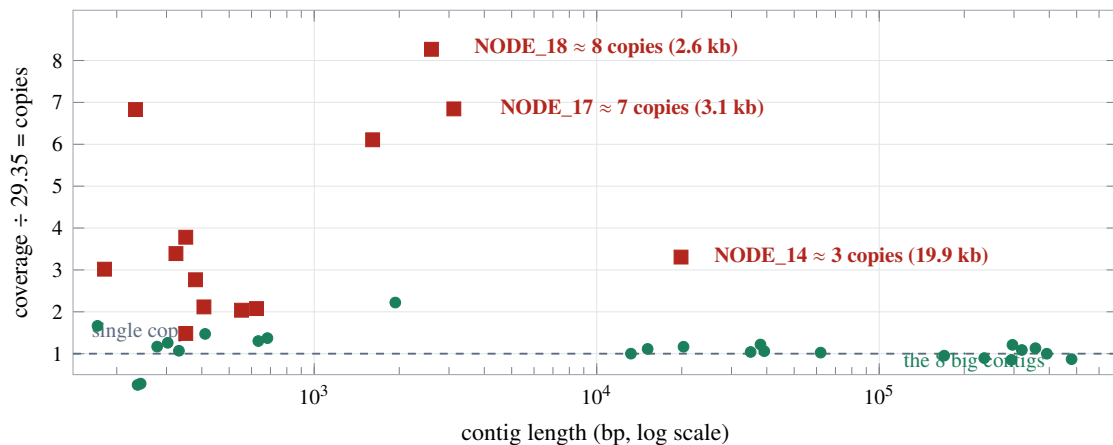


Figure 6. Every contig in `spades_assembly/contigs.fasta`, plotted by length and inferred copy number. Green circles: single copy. Red squares: collapsed repeats. The two contigs sitting below 0.3 (NODE_34, NODE_35) are low-coverage fragments, probably contaminant or chimeric.

What is actually missing from our assembly

	stored in the FASTA	present in the cell	difference
repeat sequence	32,207 bp	$\approx 124,696$ bp	≈ 92 kb collapsed

Our assembly totals 2,803,313 bp. A closed chromosome of this lineage is typically ≈ 2.75 – 2.85 Mb *including* all repeat copies. The ≈ 92 kb we are storing once instead of 6–8 times is exactly why the assembly is in 38 pieces: **the assembler cannot tell which copy connects to which neighbour.**

5.2 Counting copies does not need 71×

Contig	Length	Copies	CV at 71.6×	CV at 20×	CV at 10×
NODE_14	19,885	3	0.74 %	1.40 %	1.98 %
NODE_17	3,121	7	1.22 %	2.31 %	3.27 %
NODE_18	2,600	8	1.25 %	2.37 %	3.35 %
NODE_20	1,610	6	1.84 %	3.47 %	4.91 %

Table 2. Poisson counting precision, $CV = 1/\sqrt{k d L/\ell}$ with read length $\ell = 233$. To tell k copies from $k + 1$ you need CV well below $1/2k$; every row clears that at 10×

Action: report copy number with an interval, not a point

This directly addresses **Limitation 3** in our own METHODS_v1.7.md: “*the ϕ Sa3 boundaries are inferred from gene positions, not from attL/attR detection. The ~42.4 kb span is an estimate.*”

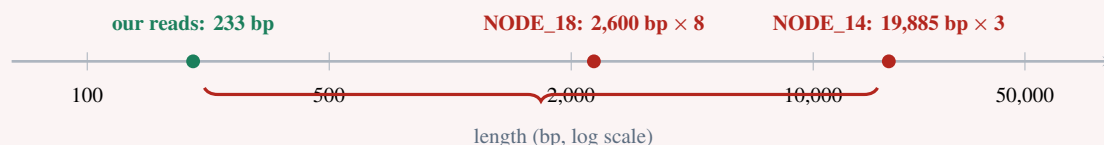
The coverage data supports a *proper interval*. The right formulation is an **integer-constrained estimate**: copy numbers must be whole numbers, and they must balance at every junction of the assembly graph (what goes in must come out). Adding those two constraints to the Poisson likelihood turns a guess into a confidence interval — and it runs in seconds on a graph this size.

6 Where mathematics provably cannot help

This section exists so that nobody on the team spends six months on a problem that has been proved unsolvable. Two of our limits are not engineering limits. They are mathematical facts.

Limit 1 — repeats longer than the read cannot be resolved. At all.

Bresler, Bresler & Tse (*BMC Bioinformatics*, 2013) proved the feasibility condition for shotgun assembly: reconstruction is possible *only if* the read length exceeds the longest interleaved and triple repeats.



No algorithm, no AI model, and no amount of coverage closes that gap. If we need a closed assembled chromosome, we need *longer reads* — roughly 20× of nanopore or PacBio, hybrid assembled with what we already have. That is a procurement decision, not a research problem.

Limit 2 — photon shot noise

From our own handbook, Chapter T6.2: for a cluster yielding N photoelectrons, $\text{SNR} \approx \sqrt{N}$. **To double the signal-to-noise ratio you need four times the photons.** This is a property of light, not of Illumina. Clusters (about 1,000 copies) already buy a factor of $\sqrt{1000} \approx 32$. There is no further mathematics to extract here.

IN PLAIN WORDS

Two of our problems are worth working on and two are not. Spending effort on cleverer algorithms to resolve the rRNA operons from 233 bp reads is like trying to read the small print in a photograph that was taken out of focus — the information is not in the file. Buy a long-read run instead.

7 After sequencing: the look-alike gene trap

The biology question: *when BLAST returns an 82 % hit to lukF-PV, is the gene there or not?*

This is the one place in our pipeline where a mathematical mistake has *already* produced a wrong biological conclusion — documented in our own METHODS_v1.7.md §4.6 as “the incorrect PVL inference in report v1.0”.

7.1 What the data actually looks like

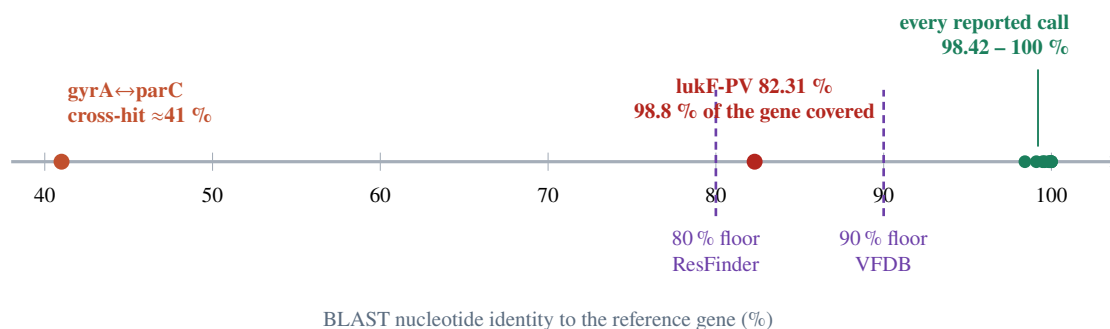


Figure 7. The identity landscape from METHODS_v1.7.md §5. The 90 % VFDB threshold was chosen *specifically* to keep the leukocidin paralogue out. It works — but it works because somebody already knew the answer.

Why the threshold is the wrong tool, even though it gave the right answer

Notice what saved us. It was **not** the 82.31 % number. It was that the same region hits hlgB — gamma-haemolysin, the known ≈80 %-identity paralogue of lukF-PV — at **99.90 %**. The reasoning was:

this looks like lukF-PV, but it looks far more like hlgB, and hlgB is definitely present.

That is a **comparison between two hypotheses**, not a comparison against a fixed number. And comparison between two hypotheses has a standard mathematical form:

$$\Lambda = \frac{P(\text{this alignment} \mid \text{the gene is really lukF-PV})}{P(\text{this alignment} \mid \text{it is really the paralogue hlgB})}$$

We computed Λ by hand, in prose, once. **It should be computed by the script, for every call, automatically.**

IN PLAIN WORDS

Percent identity throws away the most informative thing in the alignment: *which* positions differ. Two genes can both be 95 % identical to a reference and yet one of them differs only at positions that every member of the family varies at, while the other differs at the conserved catalytic residues. Percent identity scores them the same. A profile — a model of which positions are allowed to vary — does not.

7.2 Three replacements, in increasing order of effort

Method	What it does	Effort / payoff
Best-vs-second-best ratio (do this first)	For every call, also report the best hit to a <i>different</i> gene family, and the ratio. Our <code>06_pvl_test.py</code> already does this for PVL — generalise it to all five databases.	A few hours. Removes the entire class of error that produced the v1.0 mistake. Requires no new software.
Profile HMMs (HMMER)	Score against a <i>model of the family</i> rather than one sequence. Positions that are conserved across the family are weighted more heavily.	A day, plus building profiles. This is what AMRFinderPlus already uses internally.
Protein language model embeddings (ESM-2)	Represent each protein as a learned vector; orthologues cluster, paralogues separate — even at identical percent identity.	Genuinely better for hard families. See §8 for the caution that comes with it.

Why this matters beyond one gene

Our Limitations section currently has to defend a threshold. With a likelihood ratio it would not have to: the number would carry its own justification. And the same machinery covers the *gyrA*/*parC* cross-hit at $\approx 41\%$, which we currently exclude with a hand-set $\geq 90\%$ orthologue filter — another threshold that happens to be right.

8 What AI actually adds here — and one thing to be careful of

The biology question: *everyone says AI will transform genomics. For our pipeline, specifically, what does it do?*

8.1 The honest ledger

Step	What exists	Use to us
Base calling	DeepConsensus (Baid et al., <i>Nat Biotechnol</i> 2023) — transformer on raw PacBio signal; Dorado/Bonito for nanopore	~ Real but not ours: these are long-read tools. Illumina base-calling gains are now small — our own handbook §T7.4 traces Bustard → Ibis → AYB, 4.0 % → 2.8 % error.
Variant calling	DeepVariant (Poplin et al., <i>Nat Biotechnol</i> 2018)	~ Mature and excellent — but relevant only if we move from <i>assembly</i> to <i>resequencing against a reference</i> .
Gene identification	Protein language models (ESM-2; Lin et al., <i>Science</i> 2023)	✓ Directly useful. Embeddings separate orthologues from paralogues far better than percent identity — this is the §7 problem.
Genome-scale priors	Nucleotide Transformer (<i>Nat Methods</i> 2024); Evo (<i>Science</i> 2024)	~ Promising, and see the warning below.
Structure	AlphaFold2 (Jumper et al., <i>Nature</i> 2021)	~ Useful for interpreting a novel QRDR substitution; not part of routine screening.
Analysis assistance	what we already do — AI as shell and scripting assistant	✓ Already working, and our METHODS §7 states the right rule: every number traces to a deterministic re-runnable command.

What AI cannot do for us

✗ reduce photon shot noise ✗ span a 2,600 bp repeat with a 233 bp read ✗ detect a failed tile-cycle better than the 50-line script in §10

8.2 The one genuinely deep idea

Assembly and error correction are both a search for the most probable genome given the reads:

$$\hat{X} = \arg \max_X \underbrace{P(\text{reads} \mid X)}_{\text{what the machine measured}} \times \underbrace{P(X)}_{\text{what we expected beforehand}}$$

Coverage supplies the first term. **A good prior supplies the second — and a sharp enough prior substitutes for coverage.** That is why reference-guided analysis needs far less data than *de novo* assembly: the reference *is* a prior.

IN
PLAIN
WORDS

If you already know roughly what the answer should look like, you need less evidence to confirm it. That is true in the lab and it is true in mathematics. A genome language model is an attempt to supply that “roughly know” for organisms where we have no close reference.

The warning that matters specifically for AMR work

A learned prior pulls the estimate **toward what was common in its training data.**

Where the prior helps

The isolate belongs to a common, well-sampled lineage. Most of the genome is exactly what the model has seen thousands of times. The prior is right, and it saves data.

Where the prior hurts

A *novel* resistance allele. A new SCCmec junction. A rearranged plasmid.

The model has never seen it, so it will confidently smooth it into the nearest thing it has seen.

AMR research is the study of what is not yet common.

The novel findings are precisely the ones a learned prior will most reliably damage.

Our METHODS §7 already says “no result is a model assertion”. The extension worth adopting as team policy: **a learned prior is a model assertion smuggled into the data layer**. If we ever adopt AI-assisted assembly or error correction, any novel call must be re-verified against the raw reads without the model in the loop.

9 Five actions, ranked by payoff ÷ effort

#	Action	Effort	What it buys
1	Run the tile-cycle gate on every run and mask what it flags	Already written — §10. One command.	Removes 38 % of our substitution errors. Takes measured Q from 29.4 to 32.5. Do this one first.
2	Add <i>which tile, which cycle</i> as a covariate wherever we recalibrate quality	One extra term; a day	The standard recalibration (GATK BQSR) groups by reported-Q, cycle and dinucleotide context — it has no tile term at all . Ours should.
3	Report best-hit <i>versus best competing family</i> for every gene call, not just for PVL	A few hours; generalise <code>06_pvl_test.py</code>	Removes the class of error that produced the wrong PVL call in v1.0, and retires the threshold argument in our Limitations.
4	Report copy number as an interval, using integer constraints + graph balance	A day or two	Turns Limitation 3 (“the ~42.4 kb φ Sa3 span is an estimate”) into a stated confidence interval.
5	Do not buy more Illumina depth. If we need a closed genome, buy ~20× nanopore and hybrid-assemble	procurement, not research	Attacks the one limit that is genuinely information-theoretic. More short reads cannot.

The whole argument, in one picture

The problem
errors are **structured**, but the pipeline assumes they are **scattered**

The current fix
71.6× blanket coverage
— works for scattered errors, **fails completely** for structured ones

The actual fix
find the structure and remove it
50 lines, seconds of runtime, no new data

IN PLAIN WORDS

None of this says our result is wrong. It is not — the validation reproduced the published findings, and §3.4 shows the underlying data is genuinely good. What it says is that we are paying for safety in the wrong currency, and that the one fault which actually reached our data is the one kind that paying more cannot fix.

10 The drop-in script

Written and placed at `handbook_illumina/scripts/11_tilecycle_gate.py`. Standard library only, apart from samtools on PATH. Deterministic: same BAM in, same answer out.

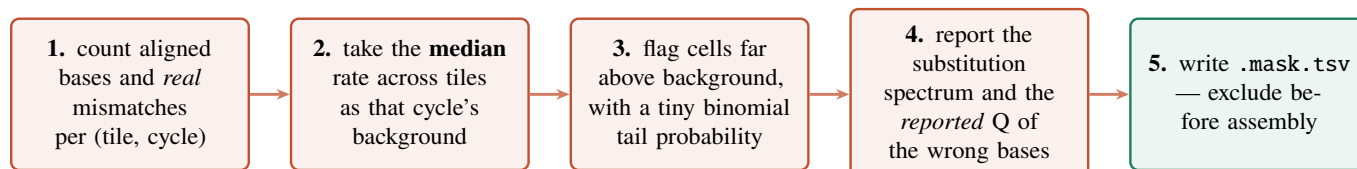
How to run it

```
# 1. map the reads back to your own assembly (once per run)
minimap2 -ax sr contigs.fasta reads.fastq.gz | samtools sort -o all_vs_contigs.bam
samtools index all_vs_contigs.bam

# 2. run the gate
python handbook_illumina/scripts/11_tilecycle_gate.py \
    all_vs_contigs.bam contigs.fasta qc/tilecycle

# 3. exclude what it flags, then re-assemble
qc/tilecycle.mask.tsv --> two columns: tile, cycle
```

10.1 What it does, in five steps



Step 2 is the only subtle one. Using the *median* across tiles rather than the mean means a handful of broken tiles cannot inflate the baseline they are being judged against. That is the whole of “robust statistics” as it applies here.

10.2 It has been tested

samtools is not installed on the machine this document was written on, so the script could not be run against our real BAM here. It was instead validated on **synthetic data with a known planted fault** — 5 tiles × 60 cycles, background error 5×10^{-4} , and one cell (tile 1103, cycle 21) forced to a 70 % error rate with all wrong bases labelled **Q37**, to reproduce exactly the failure mode we saw on our own run.

Actual output of the validation run

```
tile-cycles examined 300 (5 tiles x 60 cycles)
aligned bases 78,000
substitution rate before 0.002769 (Q25.58)
substitution rate after 0.000437 (Q33.59)

FLAGGED 1 tile-cycle(s) = 0.3333% of the run
bases discarded 260 (0.3333%)
mismatches removed 182 (84.3% of all errors)
```

tile	cycle	rate	background	excess	reported Q of the wrong bases
1103	21	0.7000	0.003846	182x	182 at Q37

It found the planted fault exactly, discarded 0.33 % of the bases, and removed 84 % of the errors. **Next step for the team: run it against the real `all_vs_contigs.bam` and confirm it recovers the four known tile-cycles (8/1101, 16/2111, 85/1113, 93/2113).** If it does, put it in the standard pipeline.

11 Where every number came from

Claim	Source file
858,085 reads · 200,631,460 bases · 71.6×	METHODS_v1.7.md §1
2,803,313 bp · 38 contigs · N50 319,865	assembly_results/quast_report/report.txt
118 gene calls from 5 databases	METHODS_v1.7.md §4.2
28,266 mismatches; 843 masked positions; 4 failed tile-cycles	handbook_illumina/results/02_empirical_errors.json
Per-tile error rates, substitution spectra, reported Q	handbook_illumina/results/02b_spike_cycles.json
Q30, trimming, over-trimming experiment (Runs A/B/C)	TUTORIAL_QC_and_Read_Cleaning.md Parts 5–7
Contig coverage $\Rightarrow$ copy number	assembly_results/spades_assembly/contigs.fasta headers
lukF-PV 82.31 %, hlgB 99.90 %, the v1.0 PVL error	METHODS_v1.7.md §4.6
Identity margins 98.42–100 %	METHODS_v1.7.md §5
φ Sa3 boundary limitation	METHODS_v1.7.md §6, item 3
SNR = $\sqrt{N}$; phasing transition matrix	handbook_illumina/theory/T6_photons_noise.md, T7_signal_statistics.md
External results cited	
Assembly feasibility condition	Bresler, Bresler & Tse, <i>BMC Bioinformatics</i> 2013
Base quality score recalibration	DePristo et al. 2011; GATK BQSR documentation
Robust low-rank + sparse separation	Candès, Li, Ma & Wright, <i>JACM</i> 2011
DeepVariant	Poplin et al., <i>Nat Biotechnol</i> 2018
DeepConsensus	Baid et al., <i>Nat Biotechnol</i> 2023
ESM-2 protein language model	Lin et al., <i>Science</i> 2023
AlphaFold2	Jumper et al., <i>Nature</i> 2021

Our data is good. Our error model was not.

51 % of the errors live in under 0.1 % of the run, and against that kind of fault, more coverage makes the answer worse. Finding the structure costs a script. Not finding it costs about 50× of sequencing, on every run, forever.